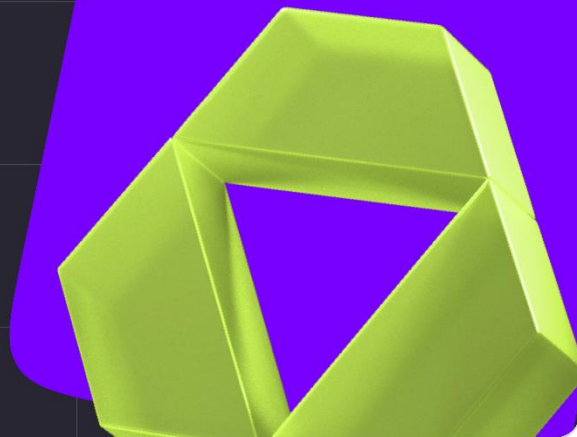


Преридер

SQL для начинающих

Занятие 12



Описание

Цель занятия — понять, как СУБД обрабатывает запросы, что такое индексы и зачем они нужны, а также научиться создавать индексы разных типов для ускорения поиска данных.

После занятия вы сможете объяснить этапы обработки SQL-запроса, создавать индексы командой `CREATE INDEX`, выбирать подходящий тип индекса и понимать, в каких случаях индекс ускоряет запрос, а в каких не используется.

Что вспомнить до занятия

● Как работает оператор `WHERE`

`WHERE` фильтрует строки таблицы по заданному условию. Без дополнительных оптимизаций СУБД просматривает каждую строку таблицы — это называется последовательным сканированием (Sequential Scan). На больших таблицах это может быть очень медленно.

```
SELECT *
```

```
FROM clients
```

```
WHERE first_name = 'Александр';
```

Этот запрос заставит СУБД просмотреть каждую строку таблицы `clients`. Если строк миллион — это миллион проверок при каждом выполнении запроса.

Что изучить до занятия

● Структуры данных: деревья и хеш-таблицы

Индексы используют специальные структуры данных для ускорения поиска. Посмотрим две основные:

Дерево (Tree)

Иерархическая структура, в которой каждый узел может иметь дочерние узлы. Поиск в сбалансированном дереве из N элементов требует примерно $\log_2(N)$ шагов вместо N . Например, в таблице с 1 000 000 строк достаточно около 20 шагов.

Хеш-таблица (Hash Table)

Структура, которая хранит пары «ключ — значение». Хеш-функция превращает ключ в число — адрес ячейки, где хранится значение. Поиск по точному совпадению выполняется за константное время $O(1)$, но диапазонный поиск и сортировка не поддерживаются.

● Понятие первичного ключа (PRIMARY KEY)

PRIMARY KEY — это особый вид уникального индекса, который вы уже использовали при создании таблиц. Он гарантирует уникальность и наличие значения в столбце и является основным средством связи таблиц через JOIN. По сути PRIMARY KEY — это UNIQUE INDEX без возможности хранить NULL.

● Этапы обработки SQL-запроса

Прежде чем запрос выполнится, СУБД проходит несколько этапов:

1. Парсинг (разбор)

Лексический разбор делит текст запроса на лексемы (токены). Синтаксический разбор проверяет соответствие правилам языка SQL. Семантический разбор проверяет, существуют ли указанные таблицы и столбцы и есть ли у пользователя права на доступ к ним.

2. Трансформация

СУБД может переписать запрос: например, подставить код представлений (VIEW) вместо их названий.

3. Планирование (оптимизация)

Планировщик строит несколько возможных планов выполнения и выбирает наиболее дешёвый по стоимости. Для сложных запросов может применяться генетический алгоритм (GEQO).

4. Выполнение

Выбранный план исполняется и возвращает результат.

● Что такое индекс и зачем он нужен

Индекс — это вспомогательная структура данных, которую СУБД хранит отдельно от таблицы. Она содержит отсортированные (или захешированные) значения одного или нескольких столбцов и ссылки на соответствующие строки таблицы на диске (tuple id).

Благодаря индексу СУБД не просматривает всю таблицу, а сразу находит нужные строки — как алфавитный указатель в конце книги. Индекс ускоряет запросы с WHERE, JOIN, ORDER BY, GROUP BY, MIN и MAX.

Обратная сторона: индексы занимают дополнительное место на диске и замедляют операции INSERT, UPDATE и DELETE, поскольку при изменении данных СУБД должна обновить и все связанные индексы.

● Синтаксис CREATE INDEX

Базовый синтаксис создания индекса:

```
CREATE INDEX idx_clients_first_name
```

```
ON clients (first_name);
```

Более полная форма с параметрами:

```
CREATE UNIQUE INDEX index_name
```

```
ON table_name
```

```
USING BTREE (column_name ASC NULLS FIRST);
```

Ключевые параметры: UNIQUE следит за уникальностью значений; IF NOT EXISTS создаёт индекс только если его ещё нет; USING выбирает архитектуру (BTREE, HASH и другие); ASC / DESC задаёт порядок; NULLS FIRST / LAST определяет, куда попадают NULL-значения.

● Виды индексов

В PostgreSQL можно создавать несколько видов индексов в зависимости от задачи:

Одноколоночный (Single-column)

```
CREATE INDEX idx_clients_first_name ON clients (first_name)
```

Составной (Multi-column)

```
CREATE INDEX idx_clients_name ON clients (last_name, first_name)
```

В составном индексе порядок столбцов важен: индекс эффективен при фильтрации по первому столбцу или по обоим сразу, но не при фильтрации только по второму.

Частичный (Partial)

```
CREATE INDEX idx_clients_email ON clients (email) WHERE email IS NOT NULL
```

Индексирует только строки, удовлетворяющие условию WHERE. Это уменьшает размер индекса и ускоряет его обновление.

Функциональный (Functional)

```
CREATE INDEX idx_email_lower ON clients (LOWER(email))
```

Индексирует результат функции от значения столбца. Ускоряет запросы вида WHERE LOWER(email) = '...'.